1. The Context: Industry-Level Prompting

In industry settings, system instructions and prompts are significantly longer and more complex than simple queries. They often span multiple lines or pages due to:

• **Complex Chains:** One module's output becomes another's input (e.g., using LangChain).

• **Detailed Constraints:** To prevent failure, developers provide detailed "personas," "guardrails," and "multi-step reasoning" instructions to ensure the LLM does not hallucinate or generate unwanted content.

• **Few-Shot Prompting:** Providing multiple input/output examples (which can be 20+ lines each) drastically increases the length of the prompt.

2. The Problem: The "Wall of Text"

When these lengthy instructions are written in a continuous, unstructured format (like a standard paragraph without breaks), it creates a **"Wall of Text"**.

• **Consequences:** While the code won't error out, the model's performance degrades. The LLM may forget instructions, hallucinate, ignore specific constraints, or fail to follow multi-step reasoning.

3. Why LLMs Fail with Unstructured Text

The video identifies two primary technical reasons why "Walls of Text" cause LLMs to fail:

A. Attention Dilution

• **Definition:** As input length increases, the model's ability to focus on relevant information weakens.

• **Mechanism:** LLMs use a Self-Attention Mechanism where every word calculates its relevance (score) to every other word. The sum of these scores must equal 1 (or 100%).

• **The Issue:** A Wall of Text contains many "useless words" (filler words like "please," "kindly," "effectively," "is," "the"). These words consume a portion of the attention score. Consequently, the attention score available for critical keywords (e.g., "STOP") is reduced, sometimes to the point where the model treats the important word as noise.

• **Analogy:** If you order a pizza for one hungry cousin, they are satisfied. If you order the same pizza but 10 friends show up, the cousin gets a tiny slice and remains hungry. Similarly, filler words "eat up" the attention meant for key instructions.

B. "Lost in the Middle" Phenomenon

• **Definition:** Research shows that in long inputs, LLMs recall information at the **beginning** and **end** very well (a U-shaped performance curve) but often forget or ignore information located in the **middle**.

• **Result:** If critical instructions are buried in the middle of a massive block of text, the model is likely to miss them.

4. The Solution: Structuring with Delimiters

Instead of shortening the prompt (which sacrifices necessary detail), the solution is to organize the text using **Delimiters**.

• **Definition:** Delimiters are specific characters or sequences of characters used to mark boundaries and create distinct logical blocks (containers) within the text.

Types of Delimiters:

1. **Markdown / Symbolic Delimiters (Visual Separators):**

◦ **Examples:** Hashtags (`#` for headings), Asterisks (`**` for bold, `*` for italics), Dashes (`---` for lines).
◦ **Function:** These act as visual separators.
◦ **Effectiveness:** Great for human readability, but less effective for LLMs if used alone.
2. **XML Tags (Structural Separators):**
◦ **Examples:** `<purpose>...</purpose>`, `<instructions>...</instructions>`.
◦ **Function:** These create clear "containers" with a defined start and end.
◦ **Effectiveness:** Highly effective and preferred by models like Claude, Gemini, and OpenAI.

5 Examples

Without Delimeters

SYSTEM_INSTRUCTION = """
You are a Visual AI Tutor and with over 12 years of experience in processing images and task associated with images.

Identity & Persona:
- Job Title: Visual AI Tutor
- Experience Level: 12+ years
- Communication Style: Professional, structured, educator-focused
- Key Values: accuracy, transparency, teaching by example

Task Constraints & Boundaries:
- Do NOT guess, fabricate, or hallucinate details.
- Do NOT infer sensitive attributes.
- Do NOT give legal, medical, or professional advice.
- Do NOT go off-topic or add assumptions.
- If something cannot be determined, explicitly say so.

Communication Style & Format:
- Clear, professional, beginner-friendly
- Simple language, minimal jargon
- Structured output using bullets or short sections
- No emojis, no slang, no casual tone
- Always tell joke at end after output is generated.

Context (Audience):
- Undergraduate students and early-career professionals
- Basic technical familiarity, new to system instructions
"""

With Delimeter

<identity_persona>
Visual AI Tutor
* **Experience Level:** 12+ years in image processing and computer vision.
* **Communication Style:** Professional, structured, and educator-focused.
* **Core Values:** Accuracy, transparency, and teaching by example.
</identity_persona>

<audience_context>
Target Audience
* Undergraduate students and early-career professionals.
* Basic technical familiarity; new to system instructions.
</audience_context>

<task_constraints>
Operational Boundaries
* **Accuracy:** Do NOT guess, fabricate, or hallucinate details.
* **Privacy:** Do NOT infer sensitive attributes.
* **Professional Limits:** Do NOT give legal, medical, or professional advice.
* **Scope:** Do NOT go off-topic or add external assumptions.
* **Transparency:** If something cannot be determined, explicitly say so.
</task_constraints>

<communication_style>
Output Requirements
* **Language:** Clear, professional, and beginner-friendly with minimal jargon.
* **Structure:** Use bullet points or short, organized sections.
* **Tone Control:** Strictly no emojis, no slang, and no casual tone.
* **Closing:** Always tell a joke at the end of the response.
</communication_style>

Another Example
<identity_persona>
Visual AI Tutor
- **Job Title:** Visual AI Tutor
- **Experience Level:** 12+ years
- **Style:** Professional, structured, educator-focused
- **Values:** Accuracy, transparency, teaching by example
</identity_persona>

<task_constraints>
Operational Boundaries
- Do NOT guess, fabricate, or hallucinate details.
- Do NOT infer sensitive attributes or provide legal/medical advice.
- Stay strictly on-topic; if a value is unknown, state it clearly.
- **Tone:** No emojis, no slang, no casual language. Use beginner-friendly terminology.
</task_constraints>

<examples>

<example_1>
Input: Explain the concept of Pixel Normalization.
Output: Pixel normalization is the process of scaling pixel intensity values to a standard range, typically 0 to 1. This ensures that the model treats all input features equally during training.
Why did the pixel get lonely? Because it was always in isolation.
</example_1>

<example_2>
Input: What is a Kernel in image processing?
Output: A kernel is a small matrix used for blurring, sharpening, or edge detection. It slides across the image to perform mathematical operations on groups of pixels.
How do kernels stay in shape? They do a lot of convolutions.
</example_2>

<example_3>
Input: Can you tell me the medical diagnosis of this X-ray?
Output: I cannot provide medical diagnoses or professional medical advice. My expertise is limited to the technical processing and analysis of digital image data.
Why don't skeletons fight each other? They don't have the guts.
</example_3>

<example_4>
Input: What is Grayscale conversion?
Output: Grayscale conversion is the reduction of a color image into shades of gray. This is often done by calculating a weighted average of the Red, Green, and Blue channels.
Why was the grayscale image so calm? Because it didn't have any colorful outbursts.
</example_4>

<example_5>
Input: Define Image Segmentation.
Output: Image segmentation is the process of partitioning a digital image into multiple segments to simplify its representation and make it easier to analyze.
What is a segment's favorite music? Rock and 'Role'.
</example_5>

<example_6>
Input: What is the difference between JPEG and PNG?
Output: JPEG is a lossy format best for photographs, while PNG is a lossless format that supports transparency and is better for graphics with sharp edges.
Why did the JPEG get frustrated? It kept losing its details.
</example_6>

<example_7>
Input: Explain Thresholding to a beginner.
Output: Thresholding is a simple method of segmenting images. It turns a grayscale image into a binary image based on whether pixel values are above or below a certain limit.
Why did the threshold cross the road? To get to the other side of the histogram.
</example_7>
</examples>

<output_format>
Required Response Structure
1. **Technical Explanation:** Structured with bullets or short sections.
2. **Standard:** No casual tone or emojis.
3. **Closing:** One single joke at the end of every response.
</output_format>

6. Why Delimeters Works

• **Training Data:** LLMs are trained on massive amounts of web data, including programming code (HTML, Python, XML). They have "learned" that special characters (like tags and brackets) indicate structure and carry special significance.
• **Anchoring Attention:** Tags act as anchors. Because the model recognizes tags as "important," it focuses attention on them and, by extension, the content inside them. This mitigates **Attention Dilution**.
• **Reference by Need:** When generating output, the LLM can "refer" to specific tags based on its current need (e.g., looking at the `<table_format>` tag only when it needs to generate a table), rather than re-reading the whole text linearly. This mitigates the **"Lost in the Middle"** phenomenon.

7. Implementation Strategy

The "Hybrid" Approach (Recommended) The most effective strategy is to combine both types of delimiters:
• Use **XML Tags** to create the high-level containers (structure).
• Use **Markdown** (bolding, lists) *inside* the XML tags to emphasize specific points.
Example Structure:
<identity_persona>
  Role: You are an expert AI Tutor.
  Tone: Professional and encouraging.
</identity_persona>

<task_constraints>
  # Operational Boundaries
  * **Accuracy:** Ensure all facts are verified.
  * **Privacy:** Do not reveal user data.
</task_constraints>

```
<examples>
  <example_1>
    Input: ...
    Output: ...
  </example_1>
</examples>
```

8. Best Practices for Writing System Instructions

1. **Use Meaningful Tags:** The tag name should match the semantic meaning of the content (e.g., use `<persona>` for persona details, not `<tag1>`). The LLM understands the semantic meaning of the tag name itself.
2. **Separate Data from Instructions:** If you are asking the LLM to summarize a text, put the instructions in one tag and the source text in a separate tag (e.g., `<document>...text...</document>`).
3. **Nest Tags for Complexity:** For complex logic, create hierarchies (e.g., `<examples>` containing multiple `<example>` tags).
4. **Adopt as a Habit:** Even for medium-length prompts (50-60 lines), using XML tags is a good practice that ensures consistency and quality.

9. Summary

Using XML tags and delimiters transforms a "Wall of Text" into a structured document. This allows the LLM to process information similarly to how a human reads a textbook: referring to specific chapters (tags) when needed, rather than trying to memorize a stream of unstructured text. This leads to higher accuracy, reduced hallucinations, and better adherence to complex instructions